E. Semaphore · ปัญหาคลาสสิก · Monitor · Java Synchronization (OSNo06 ท้าย ๆ) 🔥

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: Semaphore (ตัวนับควบคุมคิว) = เครื่องมือที่ OS ให้มาจัดคิวเข้าถึงของแชร์อย่างปลอดภัย มีคำสั่ง wait (หยิบสิทธิ์) / signal (คืนสิทธิ์) · Monitor = กล่องควบคุมระดับสูงที่ล็อกให้อัตโนมัติ

ความสำคัญกับข้อสอบ: โค้ด Bounded Buffer / Readers-Writers (ปัญหาคนอ่าน-คนเขียน) / Dining Philosophers (ปัญหานักปราชญ์กินข้าว) ต้องจำค่าเริ่มต้นให้แม่น (mutex=1, empty=n, full=0) + จุดหลอก "สลับลำดับ wait → deadlock (การติดตาย (รอกันตลอดกาล))" — ออกถี่มาก

ใช้จริงในคอมพิวเตอร์: คิวงานเครื่องปริ้นต์, connection pool ของฐานข้อมูล, ปุ่ม like ที่คนกดพร้อมกันหลายพันคน — ระบบจริงทั้งหมดใช้ semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง)/mutex คุมจังหวะ

E1. Semaphore — wait / signal (อุปมาโหลหิน 🪨)

```
wait(S) {                // P — "หยิบหินจากโหล"
    while (S <= 0);      // โหลว่าง → ยืนรอ (busy wait)
    S--;
}
signal(S) {              // V — "คืนหินใส่โหล" แล้วปลุกคนรอ
    S++;
}
```

- **Binary semaphore** = ค่า 0/1 เท่านั้น อีกชื่อคือ **mutex lock (การล็อก)** → ใช้กับ critical section (เข้าได้ทีละคน)
- **Counting semaphore** = ค่า 0..k → ใช้กับ resource (ทรัพยากร) หลายหน่วย (k = จำนวน resource)

🎯 กับดักจาก Quizizz ชุด 3

(1) CS ใช้ **binary** ไม่ใช่ counting (2) Semaphore การันตีได้แค่ **Mutual Exclusion** — progress/bounded waiting ขึ้นกับ implementation

E2. Implement แบบไม่มี busy waiting (waiting queue + block/wakeup)

```
/* แต่ละ semaphore มี waiting queue (แต่ละรายการเก็บ value + pointer รายการถัดไป) */
wait(S) {
    S.value--;                    // หยิบหินก่อนเลย
    if (S.value < 0) {            // ติดลบ = หินไม่พอ มีคนรอแล้ว
        เพิ่มตัวเองเข้า S.list;      // ต่อคิว
        block();                  // หลับ — ไม่เผา CPU
    }
}
signal(S) {
    S.value++;                    // คืนหิน
    if (S.value <= 0) {           // ยังมีคนรออยู่
        หยิบ P จาก S.list;
        wakeup(P);                // ปลุก 1 คนกลับ ready queue
    }
}
/* 💡 ค่า value ติดลบ = มี process รออยู่ |value| ตัว */
```

E3. ปัญหาคลาสสิก 1: Bounded Buffer 🔥 (semaphore 3 ตัว)

Semaphore	เริ่มต้น	ชนิด	หน้าที่
`mutex`	**1**	binary	ล็อก buffer เข้าแตะทีละคน
`empty`	**n**	counting	นับช่องว่าง
`full`	**0**	counting	นับช่องที่มีข้อมูล

```
/* Producer (สไลด์ p.20–21) */
while (true) {
    // produce an item
    wait(empty);   // ขอช่องว่าง
    wait(mutex);   // ล็อก buffer
    // เขียนลง buffer
    signal(mutex); // ปลดล็อก
    signal(full);  // บอกว่ามีของแล้ว
}
```

```
/* Consumer (สไลด์ p.21) */
while (true) {
    wait(full);    // มีของไหม?
    wait(mutex);
    // ดึงของออกจาก buffer
    signal(mutex);
    signal(empty); // คืนช่องว่าง
    // consume item
}
```

🎯 จุดหลอก: สลับ wait → DEADLOCK

ถ้า producer เขียน `wait(mutex); wait(empty);` — buffer เต็ม → producer ครอง mutex ค้างรอ empty แต่ consumer เข้าอ่านไม่ได้เพราะ mutex ถูกครอง → รอกันตาย · **กฎเหล็ก: เช็ค resource (empty/full) ก่อนล็อก (mutex) เสมอ** · จำค่าเริ่มต้นให้แม่น: mutex=1, empty=n, full=0

E4. ปัญหาคลาสสิก 2: Readers-Writers

กติกา: **Reader หลายตัวอ่านพร้อมกันได้** · Writer ทำงานเมื่อไรห้ามใครยุ่งเด็ดขาด · แชร์: `wrt=1, mutex=1, readcount=0 (int ธรรมดา)`

```
/* Writer — ง่ายสุด */
wait(wrt);
// เขียนข้อมูล
signal(wrt);
```

```
/* Reader — "ตัวแรกล็อก ตัวสุดท้ายปลด" */
wait(mutex);              // แตะ readcount ทีละคน
readcount++;
if (readcount == 1)       // reader แรกล็อก writer
    wait(wrt);
signal(mutex);
// อ่านข้อมูล (พร้อมกันได้หลายคน)
wait(mutex);
readcount--;
if (readcount == 0)       // reader สุดท้ายปลดล็อก
    signal(wrt);
signal(mutex);
```

🎯 จาก Quizizz 3

semaphore ที่ควบคุมการเพิ่ม/ลดจำนวน reader คือ **mutex** (ครอบ readcount) — **readcount เป็น int ธรรมดา** ไม่ใช่ semaphore · ที่คุม writer คือ wrt

E5. ปัญหาคลาสสิก 3: Dining Philosophers (นักปราชญ์ 5 คน)

ตะเกียบ 5 อัน = **resource** ใช้ semaphore 1 ตัวต่อ 1 ตะเกียบ เริ่มทุกตัว = 1 · กินได้ต้องมีครบ 2 ข้าง (ซ้าย i, ขวา (i+1)%5)

```
/* โค้ดนักปราชญ์ i (สไลด์ p.25–26) */
while (true) {
    wait(chopstick[i]);              // หยิบซ้าย
    wait(chopstick[(i + 1) % 5]);    // หยิบขวา
    // กินข้าว 🍚
    signal(chopstick[i]);            // วางซ้าย
    signal(chopstick[(i + 1) % 5]); // วางขวา
    // คิดปรัชญา
}
```

🎯 โซลูชัน naive นี้พัง

ทุกคน**หิวพร้อมกัน หยิบซ้ายพร้อมกัน** → ไม่เหลือตะเกียบว่าง → ทุกคนรอขวาเป็นวงกลม = **deadlock** (+ ไม่มี bounded waiting → starvation ได้) · แก้: จำกัดจำนวนคนนั่ง ≤ 4 / หยิบพร้อมกันเท่านั้น / ใช้ Monitor (ล่าง)

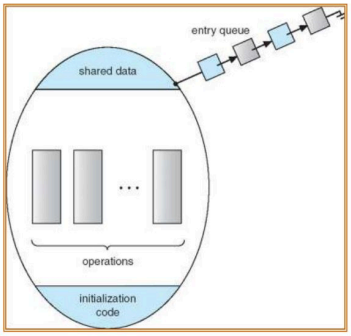
E6. Monitor — เกราะระดับสูง + Condition (เงื่อนไข) Variables

- Monitor = shared data + ฟังก์ชันรวมในก้อนเดียว (คล้าย class) — **รับประกัน mutual exclusion อัตโนมัติ: 1 process ใน monitor ต่อ 1 เวลา** (ลืมล็อกไม่ได้!)
- **Condition variable x:** `x.wait()` = สลัดล็อก หลับไปรอในคิวของ x · `x.signal()` = ปลุก 1 ตัวในคิว x

🎯 ต่างจาก semaphore ยังไง

x.signal() ถ้าไม่มีใครนอน → ***สัญญาณหายไปเฉย ๆ*** (ไม่มี counter เก็บยอด) ต่างจาก semaphore ที่ signal สะสมค่า

Schematic view of a Monitor



สไลด์ p.29 Schematic view of a Monitor: คิว entry → ใน monitor มีได้ 1 คน

Condition Variables

- condition x, y;
- Two operations on a condition variable:
  - x.wait () – a process that invokes the operation is suspended.
  - x.signal () – resumes one of processes (if any) that invoked x.wait ()

สไลด์ p.30 Condition Variables x.wait() / x.signal()

E6.1 แก้ Dining Philosophers ด้วย Monitor (กัน deadlock 100%)

```
monitor DP {
    enum {THINKING, HUNGRY, EATING} state[5];   // สถานะของแต่ละคน
    condition self[5];                          // คิวรอของแต่ละคน

    void pickup(int i) {
        state[i] = HUNGRY;
        test(i);                                // ลองดูว่ากินได้ไหม
        if (state[i] != EATING) self[i].wait;   // ยังไม่ได้ → หลับรอ
    }
    void putdown(int i) {
        state[i] = THINKING;
        test((i + 4) % 5);                      // เช็คเพื่อนบ้านซ้าย
        test((i + 1) % 5);                      // เช็คเพื่อนบ้านขวา
    }
    void test(int i) {
        // กินได้เมื่อ: เพื่อนซ้ายไม่กิน และ ฉันหิว และ เพื่อนขวาไม่กิน
        if ( (state[(i + 4) % 5] != EATING) &&
             (state[i] == HUNGRY) &&
             (state[(i + 1) % 5] != EATING) ) {
            state[i] = EATING;
            self[i].signal();
        }
    }
}
/* ใช้: dp.pickup(i) → EAT → dp.putdown(i)
   ไม่มีใคร "ถือตะเกียบเดี่ยวค้าง" → ไม่มี deadlock แน่นอน (แต่ starvation ยังเป็นไปได้) */
```

E7. Java Synchronization — synchronized (ล็อกอัตโนมัติ (คีย์เวิร์ด Java)) · wait · notify (ปลุก 1 thread ที่รออยู่)

- ทุก object มี **lock** — เรียก `synchronized` method ต้องได้ lock ก่อน · ตกคิวไปรอใน **Entry Set (คิวรอเข้าล็อก)** · ออกจาก method = คืน lock อัตโนมัติ
- `wait()` ทำ 3 อย่าง: **(1) คืน object lock (2) สถานะ blocked (3) ย้ายเข้า Wait Set (คิวหลับรอเงื่อนไข)**
- `notify()` = **สุ่ม** 1 thread จาก Wait Set → ย้ายไป Entry Set (รอแย่ง lock ใหม่ — **ไม่ได้รับทันที!** และเลือกตัวเองไม่ได้) · `notifyAll()` = ปลุก**ทุกตัว** (ปลอดภัยกว่าเมื่อ wait set มีหลายตัว)
- Java 5+ มี `Semaphore` class จริง: `acquire()` ≈ wait, `release()` ≈ signal

```
/* insert() ด้วย wait/notify (สไลด์ p.41) */
public synchronized void insert(Object item) {
   while (count == BUFFER_SIZE) {
       try { wait(); }                  // เต็ม → คืนล็อก หลับรอ
       catch (InterruptedException e) { }
   }
   ++count;
   buffer[in] = item;
   in = (in + 1) % BUFFER_SIZE;
   notify();                            // ปลุกคนที่รออยู่ 1 ตัว
}
```

```
/* remove() (สไลด์ p.42) */
public synchronized Object remove() {
   Object item;
   while (count == 0) {
       try { wait(); }                  // ว่าง → หลับรอ
       catch (InterruptedException e) { }
   }
   --count;
   item = buffer[out];
   out = (out + 1) % BUFFER_SIZE;
   notify();
   return item;
}
```

🎯 ตารางแยกพวก "wait" ที่หลอกบ่อยสุด

wait ของใคร	ทำอะไร
`wait(S)` ของ Semaphore	หยิบหิน — ลดค่า S
`wait()` ของ Java/Monitor	สลัด lock + หลับใน Wait Set
`wait()` ของ Process (บท A)	พ่อรอลูกจบ เก็บ exit status

แถม: `Thread.yield()` แค่ยอมออกจาก CPU — **ยังถือ lock ค้างอยู่!** (เหมือนล็อกห้องแล้วปีนหน้าต่างหนี)